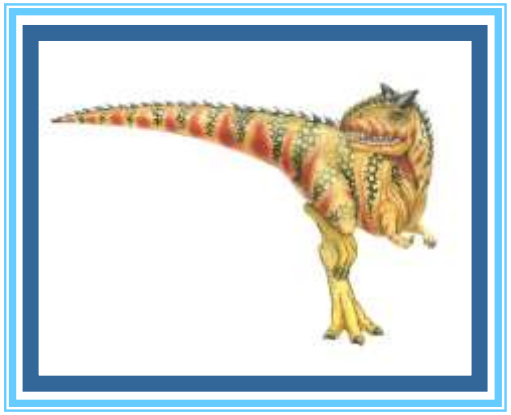
Processes

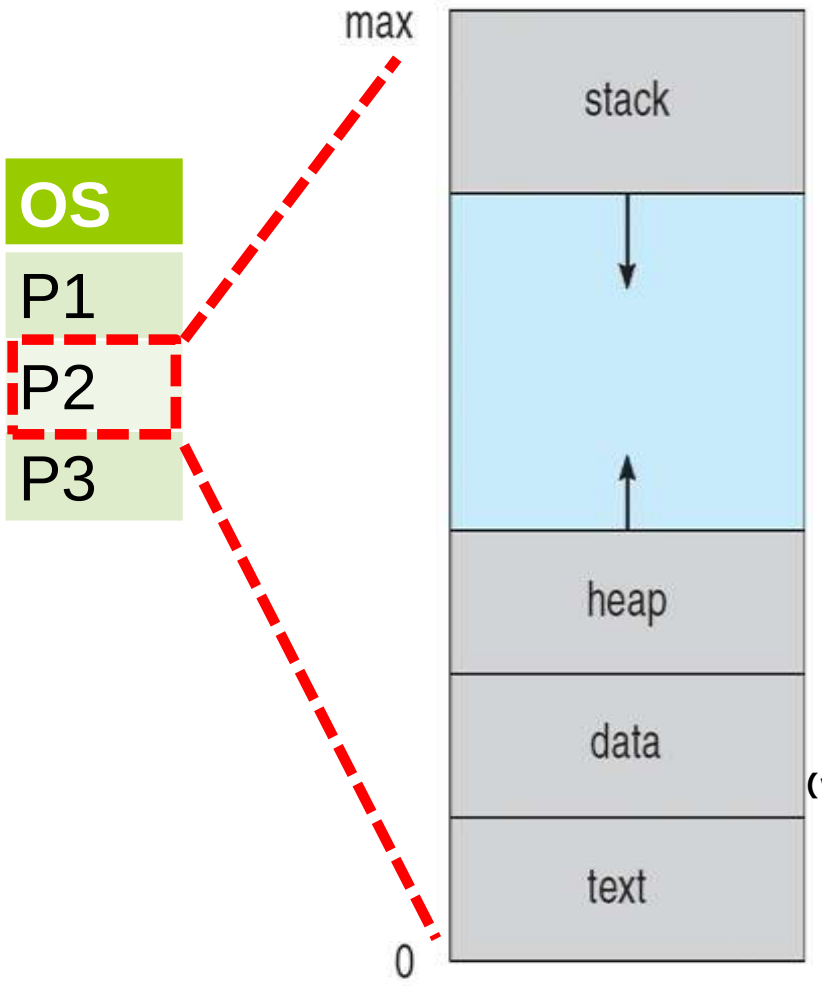


Process Concept

- ***Process*** *– A program in execution is known as process.*
- ***Program*** *by itself is not a process, it is a* ***passive entity***
- *In contrast,* ***a process*** *is an* ***active entity*** *with a* ***program counter*** *specifying next instruction to execute and a set of associated resources.*
- ***Program becomes process*** *when executable file will be loaded into memory*
- *Process execution must progress in* ***sequential fashion****.*
- *Processes can be described as either:*
  - ***I/O-bound process*** *– spends more time doing I/O than computations, many short CPU bursts*
  - ***CPU-bound process*** *– spends more time doing computations; few very long CPU bursts*

Representation of Process in Memory

A process's memory is typically divided into four main sections: text section, data section, heap, and stack.



Local variables, Function call management

Stores local variables, function parameters, and return address (Recursion)
(when is the process is terminated these variables are deleted)

Dynamic memory allocation, Free Pool of memory

Heap is for dynamic memory allocation
(The Hidden Biscuits !!)

Global and Static variables

Stores Public or Static variable
(when is the process is terminated these variables are deleted)

Compiled machine code, Read only

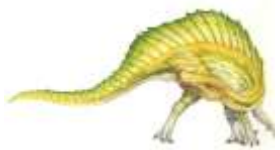
Code or Text Section (Instructions)

Process Concept

- ***text section*** → *compiled machine code instructions of the program. This is where the actual executable code of the program resides.*

  The text section is typically read-only (should not be modified during program execution.

- ***data section*** → *stores global, static variables, as well as constant data.*
- ***heap*** → *for dynamic memory allocation, and is managed via calls to new, delete, malloc, free, etc.*
- ***Stack*** → *is used for function call management and local variable storage. Each function call creates a new stack frame, containing local variables and return addresses.*



Process Concept

- *Note that the stack and the heap start at opposite ends of the process's free space and grow towards each other.*
- *If they should ever meet:*
  - *then either a stack overflow error will occur,*
  - *or else a call to new or malloc (dynamic memory allocation) will fail due to insufficient memory available.*
- *When processes are swapped out of memory and later restored, additional information must also be stored and restored.*
- *Key among them are the program counter and the value of all program registers.*

Operations on a Process by OS

- Create (Resource Allocation)

  Allocation of recourses such as Memory, I/O devices, file etc.

- Schedule, Run

  Process can be scheduled and process can be run on CPU

- Wait/Block

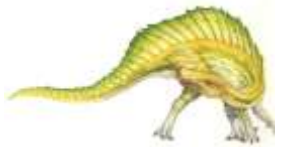
  Process can be waited and blocked.

- Suspend, Resume

  Process can be suspended and can be resumed afterwords.

- Terminate (Resource Deallocation)

  All the resources will be taken back from the process

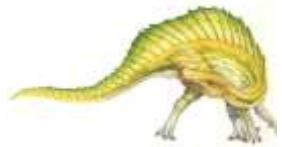


Attributes of the Process

- PID ➡ Unique Process ID given by OS
- PC ➡ Program Counter
- GPR ➡ General Purpose Register (to store the Process values if it is suspended, so that it can be resumed from where left)
- List of Devices
- Type ➡ Foreground/Background
- Size
- Memory Limits
- Priority
- State
- List of Files ➡ List of files accessed by the process

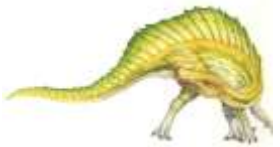
PCB

(Under the control of OS)



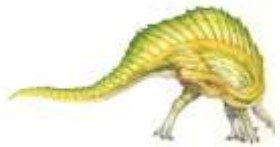
Process Control Block (PCB)

- *Each process is represented in the OS by a process control block* ***(PCB)*** *– also called a task control block.*
- *For each process, the OS maintains the data structure, which keeps the complete information about that process.*
- *This record or data structure is called* ***Process Control Block (PCB).***

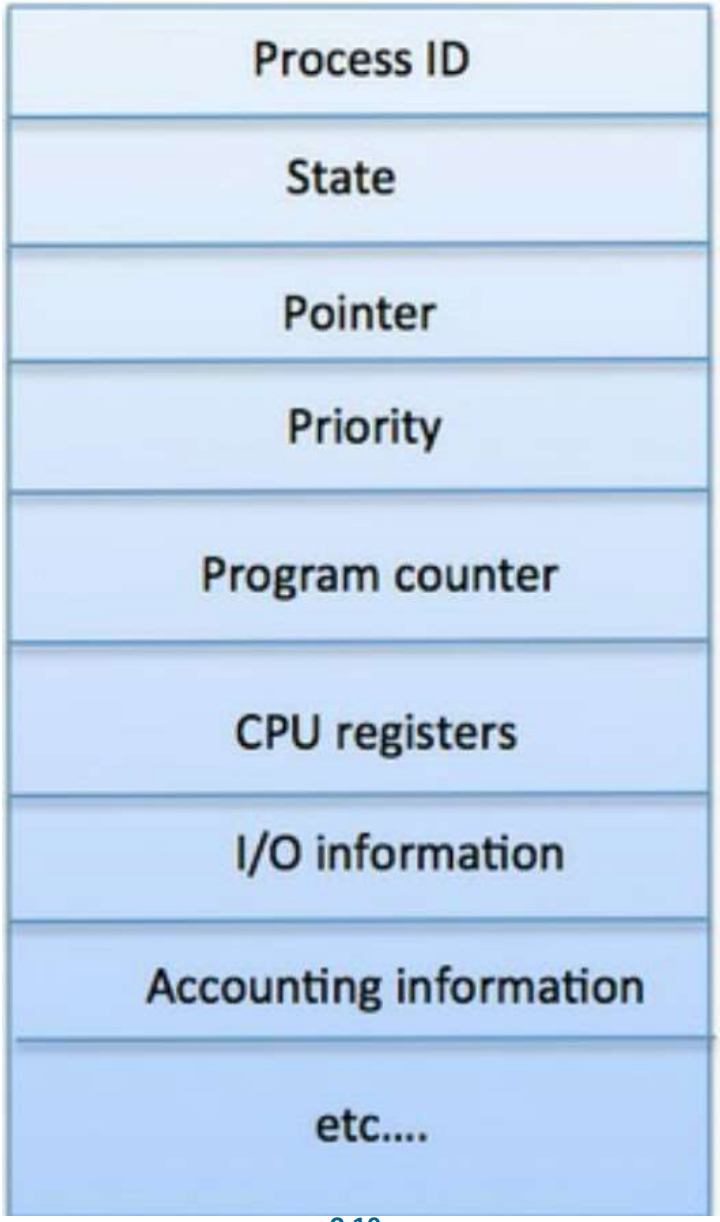


Process Control Block (PCB)

- *Whenever a user creates a process, the OS creates the corresponding PCB for that process.*
- *These PCBs of the processes are stored in the **memory** that is **reserved for the OS**.*
- *PCB keeps all information needed to track a process.*
- *After a process terminates its corresponding PCB is also removed from the system.*
- *The PCB has many fields that store the relative information about that process*



Process Control Block (PCB)



Unique Id

Current State

Points to parent process

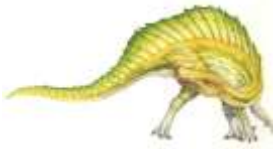
Process Priority

Points to next instruction to be executed

Context Switch

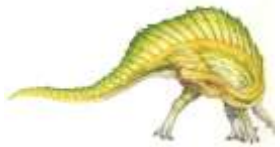
List of I/O devices allocated to process

Amount of CPU used



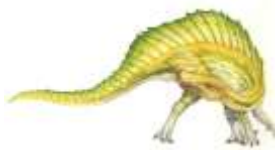
Process Control Block (PCB)

- ***Process-id:*** *Whenever a new process is created by the user, the OS allots a number to that process. This number becomes the unique identification of that process*
- ***Process State:*** *A process in its lifetime undergoes different states. Like, a process may be in* ***waiting state, running state, ready state, blocked state, halted state****, and so on.*
- ***Process Priority:*** *Priority specifies how urgent is to execute the process. It is a* ***numeric*** *value, lesser the value, greater is the priority of that process. The priority of the process can be assigned externally by the user or by the OS itself.*



Process Control Block (PCB)

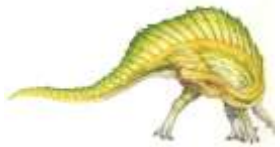
- ***Process Accounting Information:*** *This field of PCB gives the account/description of the resources used by that process. Like, the amount of CPU time, real-time used, connect time.*
- ***Program Counter:*** *The program counter is the pointer to an instruction in the program or code that is to be executed next. This field contains the address of the instruction that will be executed next in the process.*
- ***List of Open Files:*** *It contains the information of all the files that is required by the program during its execution.*



Process Control Block (PCB)

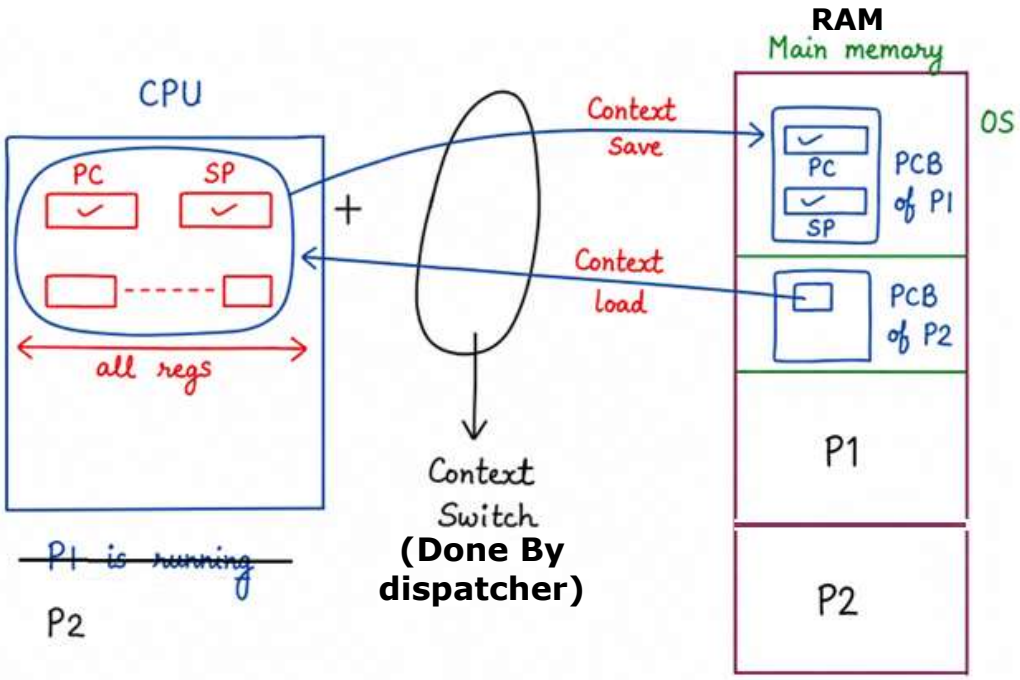
- ***Process I/O status Information:*** *Sometimes the process executing in the system require I/o devices. So, this field of PCB contains the list of all the I/O devices allocated to the process during its execution.*
- ***CPU Registers:*** *Whenever an interrupt occurs and there is a context switch between the processes, the temporary information is stored in the registers. So, that when the process resumes the execution it correctly gains from where it leaves. CPU registers are used to hold those temporary values or information.*

Let us understand what is
"context" and
"context switch"?



Process Control Block (PCB)

- *When CPU switches to another process, the system must **save the state** of the old process and load the **saved state** for the new process this task is known as **context switch.***
- ***Context** of a process represented in the PCB*
- *Context-switch time is overhead; the system does no useful work while switching*

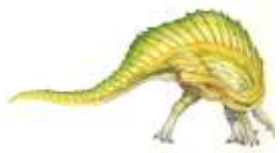


Process Control Block (PCB)

- ***PCB Pointer:*** *In this field, the pointer has an address of the next PCB, whose process state is* ***ready****. In this way, the operating system maintains the hierarchy of all the processes.*

Note:

- *PCB of each process resides in the main memory.*
- *There exists only one PCB corresponding to each process.*
- *PCB of all the processes are present in a linked list.*



QnA

While running a process can access its PCB from main memory?

Soln: PCB's are stored in OS protected area, No process can access this even the process itself

A process in the context of computing is:

a) A set of instructions to be executed on a computer
b) A program in execution
c) A piece of hardware that executes a set of instructions
d) The main procedure of a program

Which technique was introduced because a single job could not keep both CPU and IO devices busy?

a) Real Time
b) Spooling
c) Preemptive Scheduling
d) Multiprogramming

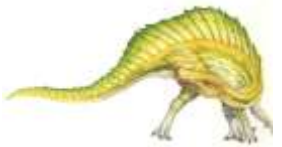
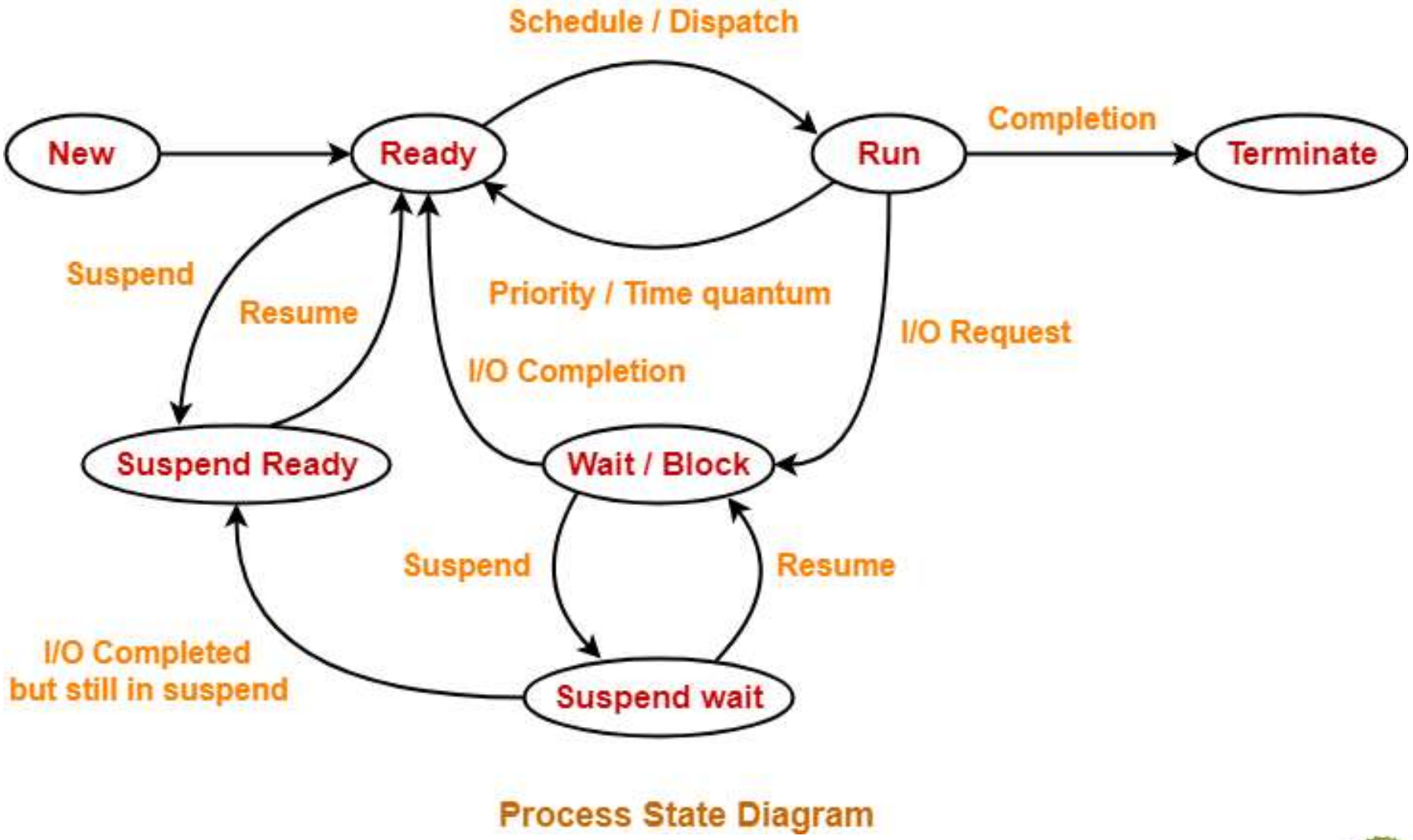


Diagram of Process State

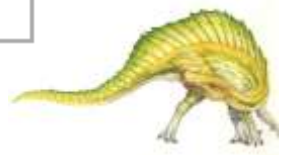


Process State Diagram

Let's all draw this and then understand it step by step…

Diagram of Process State

State	Present in Memory
New state	Secondary Memory
Ready state	Main Memory
Run state	Main Memory
Wait state	Main Memory
Suspend wait state	Secondary Memory
Suspend ready state	Secondary Memory
Terminate state	–

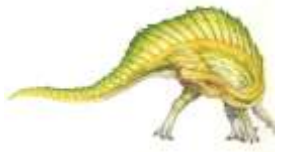


Process State

- As a process executes, it changes state.
- The state of the process is defined by the current activity of that process.
- Each process may be in one of the following states:-
- ***New***
- ***Ready***
- ***Running***
- ***Waiting***
- ***Terminated***

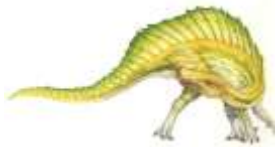
Except of these states there are two more states. They are

- ***Suspended Ready***
- ***Suspended Wait.***



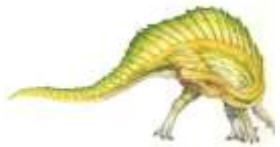
Process State

- ***New:-*** *A process is said to be in new state when a* ***program present*** *in the* ***secondary memory*** *is initiated for execution.*
- ***Ready:-*** *A process moves from new state to ready state after it is loaded into the main memory and is ready for execution.*
  - *In ready state, the process waits for its execution by the processor.*
  - *In multiprogramming environment, many processes may be present in the ready state.*
  - *After the successful creation of a process, it is placed in Ready state by Long Term Scheduler.*
  - *In this state, process will wait to be assigned to a processor.*



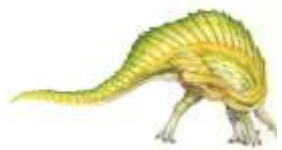
Process State

- ***Running:-*** *A process moves from ready state to run state after it is assigned the CPU for execution.*
  - *From the Ready state one of the process is selected and it will be scheduled or Dispatched to the Running State.*
  - *If a process is in Running state that means instructions of the process are being executed*
- ***Waiting:-*** *A process moves from run state to block or wait state if it requires an I/O operation or some blocked resource during its execution. After the I/O operation gets completed or resource becomes available, the process moves to the ready state.*
- ***Terminated:-*** *The Process has finished execution.*



Process State

- ***Suspended Ready:-*** *A process moves from ready state to suspend ready state: if a process with higher priority has to be executed but the main memory is full. Moving a process with lower priority from ready state to suspend ready state creates a room for higher priority process in the ready state.*
- *The process remains in the suspend ready state until the main memory becomes available.*
- *When main memory becomes available, the process is brought back to the ready state.*
- ***Suspended Wait:-*** *When resources are short enough to manage the process in wait/block state then some of the process will be suspended and moved to suspended block state.*
- *Whenever the resources are sufficient the process are resume back to the wait state by MTS.*
- *When the Process is in Suspended Ready or Suspended wait, it reside in secondary memory or Backing Store.*



Schedulers

- Needed Because ?

  For resource utilization.

- Scheduling Queues

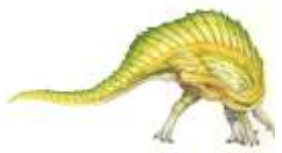
  - Job Queue: All processes which are in the new state.

  - Ready Queue: All the processes which are in ready state.

  - Device Queues: All those processes which are waiting for a specific device

- *Schedulers are special **system software** which handle process scheduling in various ways. Schedulers are of three types −Types of scheduler*

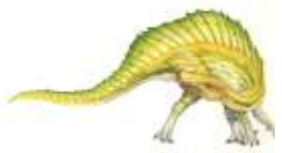
1. ***Long term scheduler***
2. ***Short term scheduler***
3. ***Mid term scheduler***



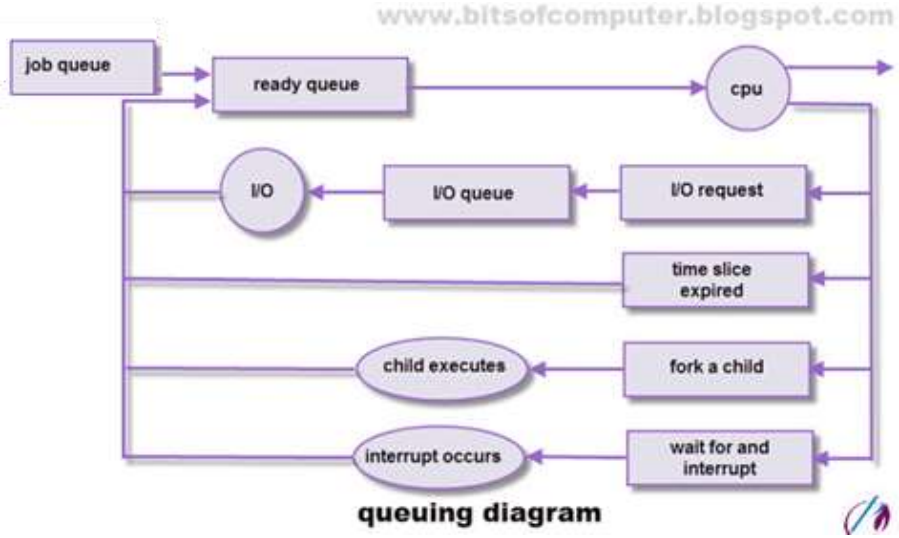
Scheduling Queues

- Maintains **scheduling queues** of processes
  - ***Job queue*** – As processes enter the system they put into the job queue, which consist of all processes in the system.
  - ***Ready queue*** – set of all processes residing in main memory, ready and waiting to execute (stored as a ***linked list).***

    A ready-queue header contains pointers to the first and final PCBs in the list. PCB include a pointer field that points to the next PCB in the ready queue.
  - ***Device queues*** – The list of processes waiting for a particular I/O (like tap, disk etc..) device is called a device queue. Each device has its own device queue.
  - During life cycle processes migrate among the various queues



Representation of Process Scheduling



Squares - queues

Circles - recourses

- *A new process is initially put in the job queue. It waits in the ready queue until it is selected for execution.*
- *Once the process is assigned to the CPU and is executing, one of several events could occur:*

1. *The process could issue an I/O request, and then be placed in an I/O queue.*
2. *The process could create a new sub process and wait for its termination.*
3. *The process could be removed forcibly from the CPU, as a result of an interrupt, and be put back in the ready queue.*

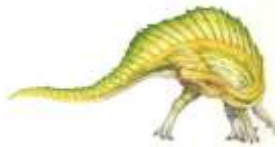
A process continues this cycle until it terminates, at which time it is removed from all queues and has its PCB and resources deallocated.

Process Scheduling

- *CPU scheduling is the basis of multiprogrammed operating systems. By switching the CPU among processes, the operating system can be more productive.*
- ***Which process gets to run next?.***

  Process scheduling is determining which process moves from ready state to the running state."

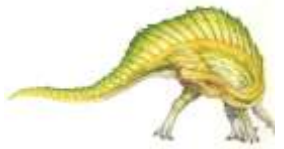
  The part of the operating system concerned with this decision is called the ***scheduler****, and algorithm it uses is called the* ***scheduling algorithm.***



Process Scheduling

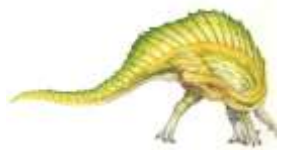
The events where scheduler has to come in act to take decision are as follow:

1. *When current process get **terminated**.*
2. *The current process goes form the **running to the waiting state**, because it issues an I/O request or some operating system request can't be satisfied immediately.*
3. *A **timer interrupt** makes the scheduler to run process some allotted interval of time and to move process from running to the ready state.*

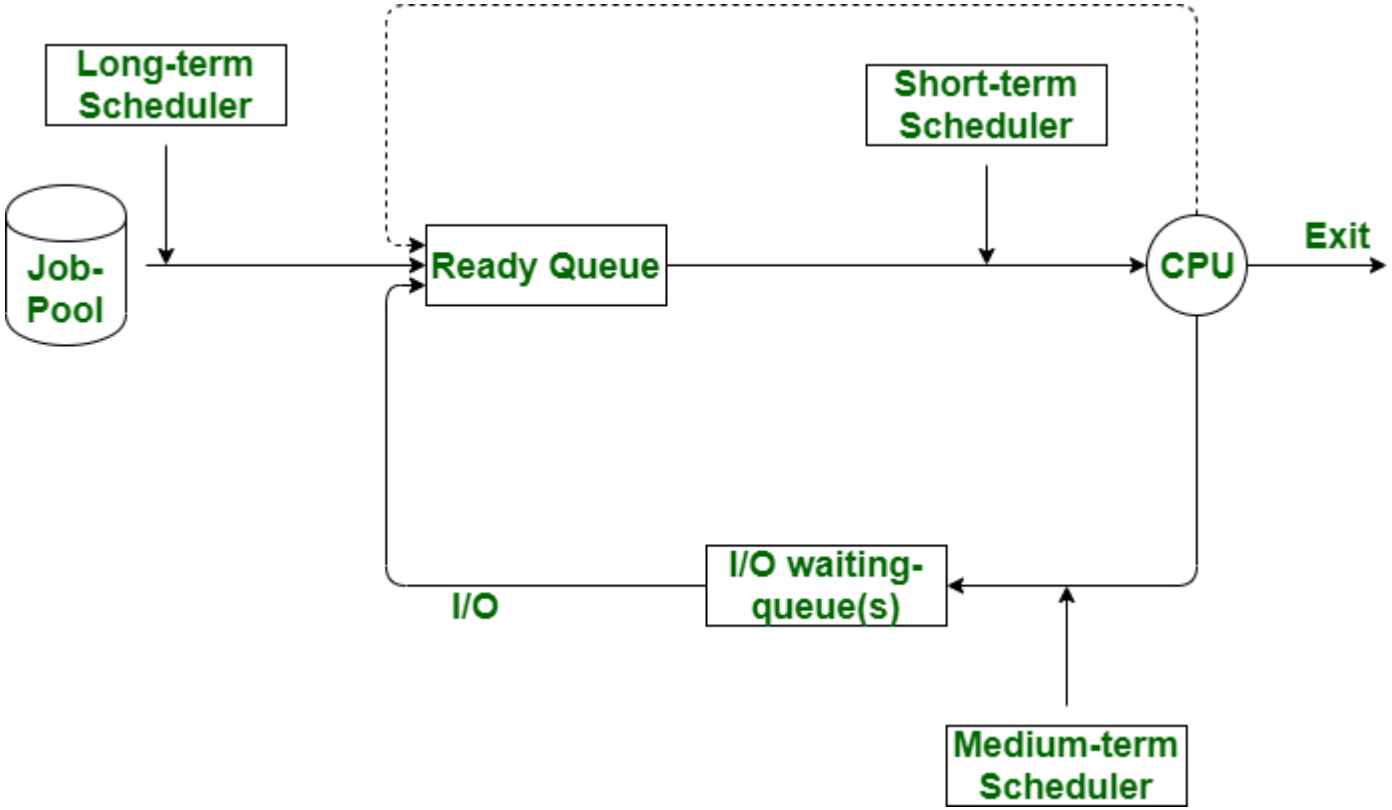


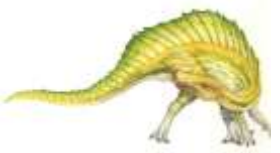
Types of Schedulers

- ***Long-term scheduler*** *(or* ***job scheduler****) – selects which processes should be brought into the ready queue*
  - *Long-term scheduler is invoked infrequently (seconds, minutes) ⇒ (may be slow)*
  - *The long-term scheduler controls the* ***degree of multiprogramming***
- ***Short-term scheduler*** *(or* ***CPU scheduler****) – selects which process should be executed next and allocates CPU*
  - *Short-term scheduler is invoked frequently (milliseconds) ⇒ (must be fast)*
- ***Medium-term scheduler:*** *Remove process from memory, store on disk, bring back in from disk to continue execution:* ***swapping***



Types of Schedulers





Thank You